

# XinoIA

Study guide: how to set it up, what you can do with it, and how to work with a team of AI agents

---

Covers version 0.1.4, for Windows 10 and 11.

Project: github.com/HelioASjunior/agents-ia

XinoIA is an independent project, not affiliated with or endorsed by Anthropic, OpenAI, Google or Epic Games.

The interface is in Brazilian Portuguese for now, so button names appear here in Portuguese, with their meaning in English.

# Contents


**1.** What XinoIA is, and the main concepts

**2.** Install, update and uninstall

**3.** Connect the AIs: browser, API key and multiple accounts

**4.** Projects and project memory

**5.** Agents and permissions

**6.** Chat

**7.** Missions

**8.** Workflows

**9.** Execution modes

**10.** Following the work: office, terminal, history and dashboard

**11.** Integrations, MCP and Unreal Engine

**12.** Usage: how to spend less of your limit

**13.** Moving a project into XinoIA

**14.** Security and privacy

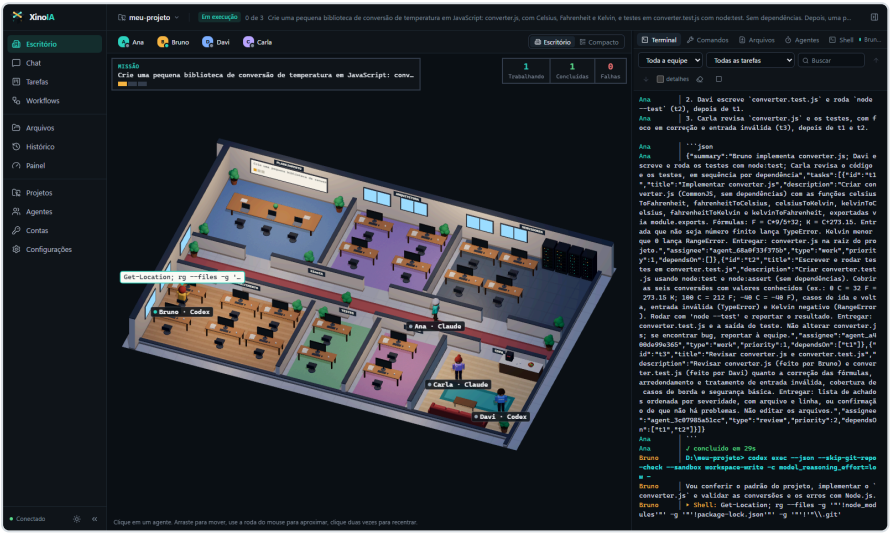
**15.** Troubleshooting

**16.** Hands-on path: from zero to a team

# 1. What XinoIA is, and the main concepts

XinoIA runs AI coding tools, such as Claude Code and Codex CLI, as one team on your computer. You give it a goal. A coordinator plans the work, each agent takes a task and hands the result to the next, and another agent reviews it.

It isn't an AI and doesn't include one: **it drives the official tools** you install, with your own accounts. There is no XinoIA server, sign-up or telemetry.




A real mission: the 3D office on the left and the live terminal on the right.

## Vocabulary

| Term | What it is |
|---|---|
| **Tool** | A vendor's official CLI, such as Claude Code or Codex CLI. |
| **Account (profile)** | A login or an API key for a tool. Each separate profile has its own folder. |
| **Project** | The folder where the agents work. |
| **Agent** | A team member: a role, a tool, an account, a model and a permission level. |
| **Run** | One round of work by an agent, with everything it did on record. |
| **Mission** | A goal split into tasks, with dependencies, reviews and a final report. |
| **Workflow** | A process you design, which can be run as often as you like. |
| **Integration** | Skills, plugins and MCP servers an agent can use. |

> **A subscription isn't unlimited**
> Your plan's limits still apply. A team spends more than a single conversation, because each task is a whole piece of work.

# 2. Install, update and uninstall

## Install

1. Open `github.com/HelioASjunior/agents-ia/releases` and download `XinoIA-Setup-vX.X.X.exe`.
2. Run it. The installer doesn't have a Microsoft certificate yet, so Windows may show a SmartScreen warning: choose **More info**, then **Run anyway**.
3. Open **XinoIA** from the Start menu. The wizard shows the AIs, connects your account, picks the folder, creates a starter team and runs a real test.

**Portable version:** `XinoIA-Portable-vX.X.X.exe` runs without installing and keeps its data in `XinoIA-dados`, next to it. It doesn't update itself.

## Update

In **Configurações > Atualizações** (Settings > Updates), XinoIA checks when it opens and every 4 hours, if automatic checks are on. Nothing is installed until you ask. Before installing, it verifies the package signature and backs up your data.

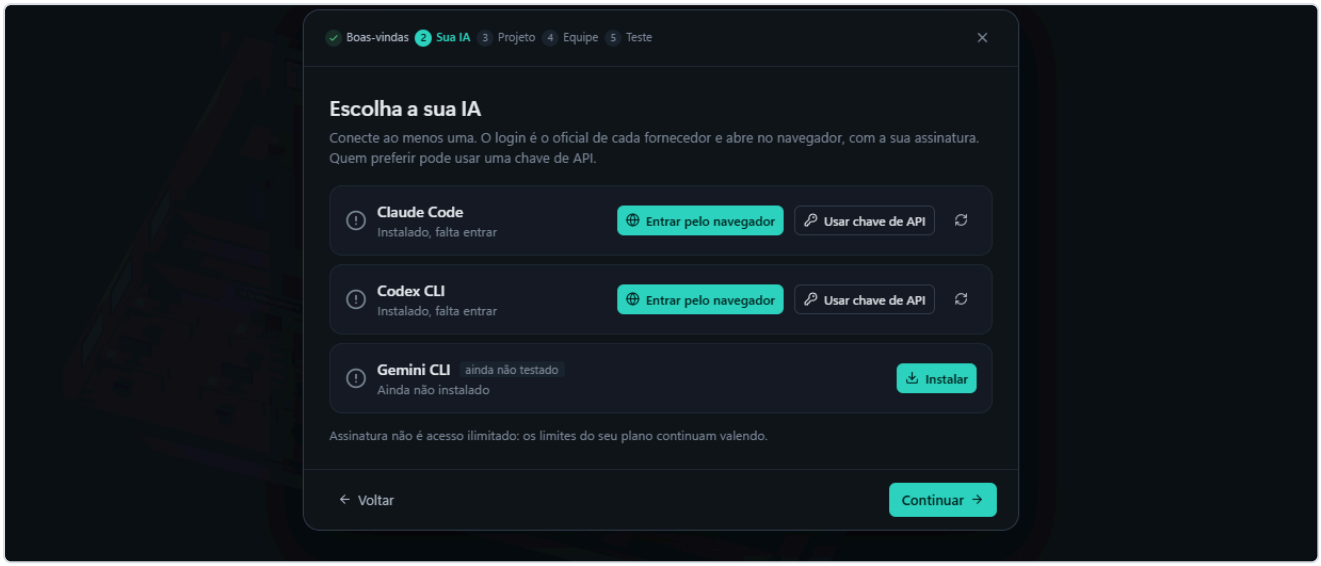
| With work in progress | What happens |
|---|---|
| **Instalar quando terminar** (install when finished) | Waits for the work to end, including between the tasks of a mission, and only then restarts. |
| **Interromper e reiniciar** (interrupt and restart) | Stops the agent right away. The mission is paused, and on resume the task starts over, using quota again. What it already changed stays in the files. A mission still in planning must be created again. |
| **Lembrar depois** (remind me later) | Does nothing for now. |

## Where your data lives

In `%APPDATA%\XinoIA`, outside the program folder. Updating or uninstalling deletes nothing. To delete everything, remove that folder after uninstalling. Backups are in `backups`, and logs in `logs`. You can create a backup right away in Configurações > Dados (Data).

> **Practice**
>
> Open Configurações > Dados and find the data folder. Then, under Atualizações, click **Verificar atualizações** (check for updates) and see the installed version.

# 3. Connect the AIs




Choosing the AI in the wizard, as a new user sees it.

## Install the tool

The **Instalar** (install) button runs the official installer in plain view. The Claude Code one doesn't need Node.js. The Codex CLI and Gemini CLI ones do. Installed it yourself? Click **Verificar tudo** (check all) in Contas (Accounts).

## Sign in through the browser (subscription)

Click **Entrar pelo navegador**. The official page opens, and you sign in as usual, including with Google if your account uses it. The window closes by itself when sign-in finishes. If the browser doesn't open, use **Abrir a página de novo** (open the page again). If the tool asks something, or the browser shows a code, click **Ver terminal** (see terminal).

## Use an API key

This is for pay-as-you-go: each use is billed by the vendor, separately from a subscription. The key lives in its own profile, encrypted by Windows and tied to your user, and never returns to the screen. For now, this works only on Windows. A backup restored on another computer can't read the key.

## Multiple accounts

In Contas, use **Adicionar conta** and **Entrar em outra conta pelo navegador**. Check that the right account is open in the browser before confirming: if it's still signed in to the first one, both profiles may end up on the same account.

> **Use only your own accounts**
> Follow each vendor's terms. XinoIA never switches accounts by itself to get around a limit: each agent stays on its account. The only optional automatic handover is passing the task to an agent from **another provider**.

| Tool | Subscription | API key | Status |
|---|---|---|---|
| Claude Code | Yes | Yes | Tested with a real account |
| Codex CLI | Yes | Yes | Tested with a real account |
| Gemini CLI | Google sign-in | Yes | **Not tested yet** |
| Others (Contas > Outras ferramentas) | Depends on the tool | | Must read the prompt from standard input |

# 4. Projects and project memory

In **Projetos**, add a **Pasta existente** (existing folder), a **Pasta nova** (new folder), or use **Clonar do Git** (clone). Agents work in that folder, on whichever branch is open in it.

> **Check the project before running**
> The selector at the top of the screen shows which project you're in. Look at it before starting a mission. In real projects, use Git and commit first: that's what lets you undo anything.

## Project memory

In the **Memória do projeto** icon, record short notes and decisions. Everything there is passed to the agents in this project, and decisions they mark are added too. Use it for what always applies, such as "all interface text is in Portuguese".

## Rule files

Each tool reads its own file in the project folder: Claude Code reads `CLAUDE.md`, and Codex reads `AGENTS.md`. If you only have one, create the other pointing to it, or part of the team won't see the rules.

> **Practice**
> Create a project with **Pasta nova**, open the project memory and record a decision. Then ask an agent what that rule is.

# 5. Agents and permissions

In **Agentes**, each one has a name, color, role, tool, account, model, effort, permission, extra instructions and integrations.

- **Role:** coordination, architecture, development, frontend, backend, Unreal Engine, code review, testing, design, documentation or custom. It sets the room in the office and how the agent receives work.
- **Model:** empty uses the tool's default. In Codex, the list comes from the installed version's catalog, with the effort levels each model accepts. In Claude, pick a fixed version, like **Opus 5.5 (claude-opus-5-5)**, or an alias, like **opus**, which follows the latest one. The agent card shows the model used in the last run. Not every account can use every model. In **Outro...** (other), you can type a name that isn't listed.
- **Extra instructions:** rules for this agent only.

## Permissions: the most important decision

| Level | What it can do | In practice |
|---|---|---|
| **Somente leitura** (read only) | Reads the project. Doesn't change files, run commands or use MCP tools. | Ideal for coordinating and reviewing. It can't run `git status` or `git diff`: put that information in the task description. |
| **Editar** (edit) | Changes project files. In Codex, runs commands in the protected area. In Claude, terminal commands stay blocked. | To run tests, use Codex. The protected area doesn't write outside the project folder. |
| **Total** (full) | Runs any command, with your account's access to the computer. | Works only in the Autônomo controlado mode. Use it only on the agent that needs it, with a clean Git state first. |

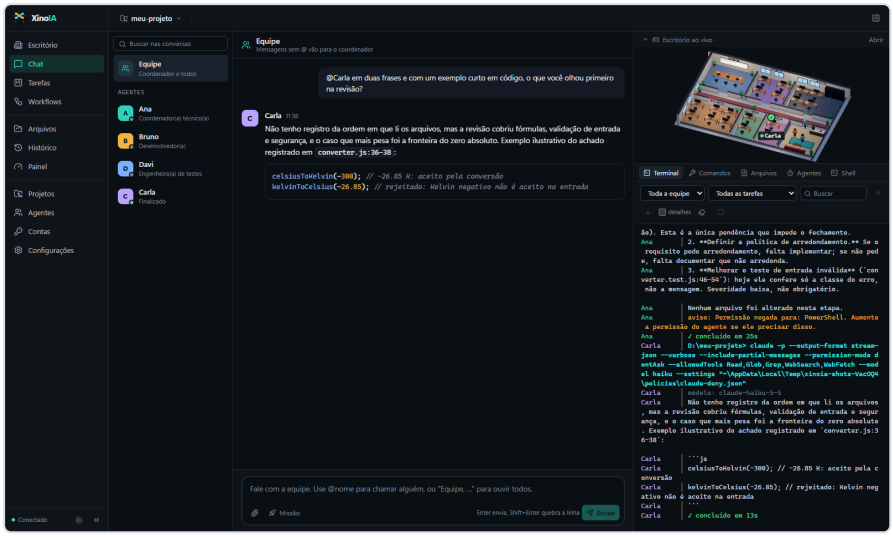
> **When "edit" isn't enough**
> Programs that write outside the project folder fail under "edit". In a real project, the Unreal build stopped when trying to write to `%LOCALAPPDATA%\UnrealBuildTool`. The protected area may also deny commands such as `tasklist`. The way out is "full", and that decision is yours.

## A team to start with

| Agent | Role | Tool | Permission |
|---|---|---|---|
| Coordination | Coordination | Claude Code | Read only |
| Implementation | Development | Codex | Edit |
| Testing | Testing | Codex | Edit |
| Review | Code review | Claude Code | Read only |

A reviewer on another tool brings a second opinion: an error one AI misses, the other may catch.

# 6. Chat




Team chat, with the small office view above the terminal.

| How to write | Who answers |
|---|---|
| No mention, in the Equipe (team) channel | The coordinator. |
| `@name` | That agent. |
| Starting with `Equipe, ...` | Everyone analyzes in read mode, and the coordinator decides last. It uses more of your limit. |
| In an agent's channel | A direct conversation, which continues where it stopped, even after a restart. |

You can attach files, search conversations and turn on **Missão** in the team chat, so your request becomes a mission. With a mission paused, what you write goes into its timeline as guidance.

> **Practice**
>
> Ask `@Review what would you look at first in this project?` and watch the run appear in the terminal beside it.

# 7. Missions

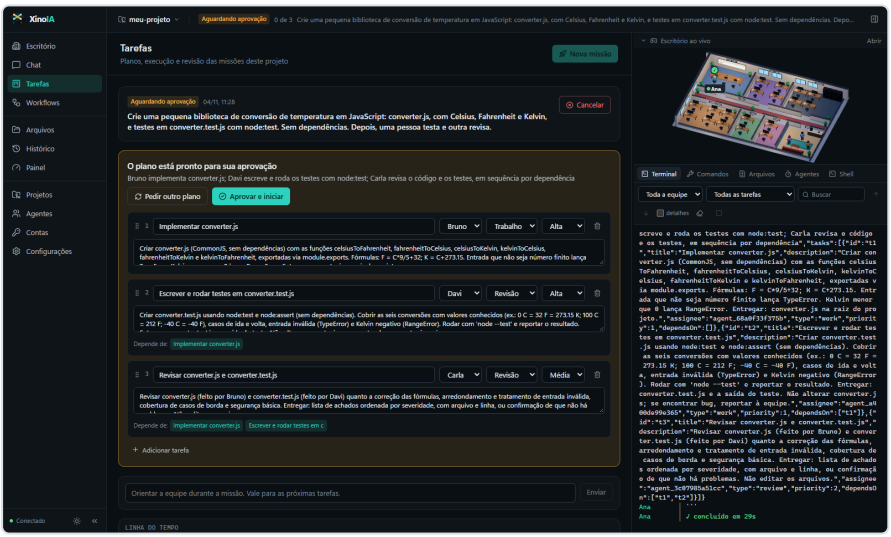
A mission turns a goal into tasks. It's the heart of XinoIA.

## Create

In **Tarefas > Nova missão** (Tasks > New mission), write the **Objetivo** (goal), pick the team and choose:

- **Coordenador decide:** builds a plan for you to approve. **Sequência fixa:** each agent, in the marked order.
- **Tarefas ao mesmo tempo** (tasks at once): above 1, in a Git project with at least one commit, each task works in its own worktree. Without Git, they run one at a time.
- **Rodadas de correção após reprovação** (fix rounds after rejection): how many times work goes back before stopping and calling you.
- **Hand over to another provider:** optional, if an account hits its limit. It never switches to another account of the same provider.

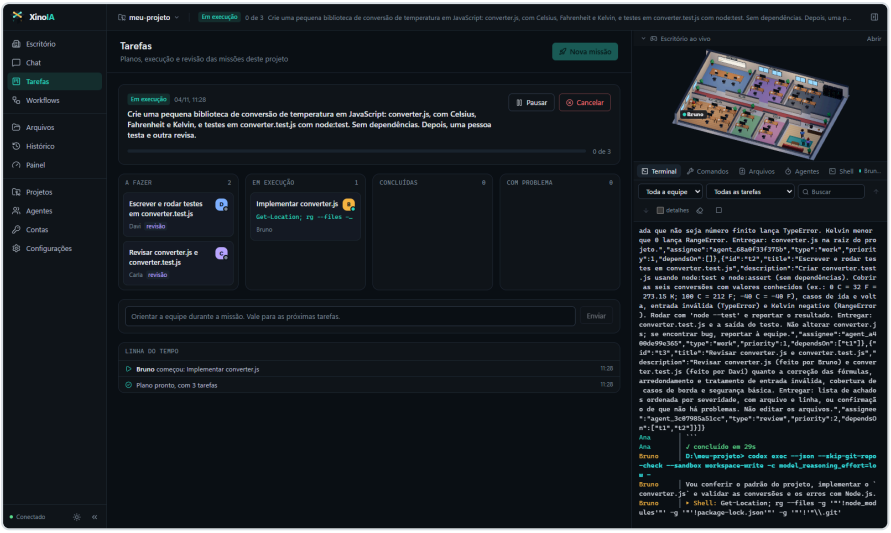
## Approve the plan




A real plan. You can edit each task, ask for another plan or approve.

Edit the title, description, assignee, type and priority. Use **Pedir outro plano** (ask for another plan), saying what to change, or **Aprovar e iniciar** (approve and start).

## While it runs




The task board: to do, running, done and with problems.

| Action | What for |
|---|---|
| **Pausar** (pause) | Stops starting tasks. What's already running finishes normally. |
| **Retomar** (resume) | Continues a paused mission. In supervised mode, it releases the next task. |
| **Tentar de novo** (retry) | On a task: redoes one that failed, or a review rejected at the fix limit. |
| **Pular** (skip) | On a task: moves on without it. It's your decision, and the next tasks can then run. |
| **Reatribuir** (reassign) | Passes an unfinished task to another agent. |
| **Orientar a equipe** (guide) | Your message applies to the next tasks. |
| **Cancelar** (cancel) | Ends the mission and the runs in progress. |

## Review and rejection

A review ends with a verdict: approved or rejected. When rejected, XinoIA creates a fix task and a new review by itself, and what came next now depends on it. At the round limit, the mission pauses.

> **Resuming doesn't approve**
> A review rejected at the limit holds back what comes next: **Retomar** makes the mission stop again, with an explanation. Open the task and choose **Tentar de novo** or **Pular**. A review without a clear verdict counts as a failure, never as approved.

## Parallel work

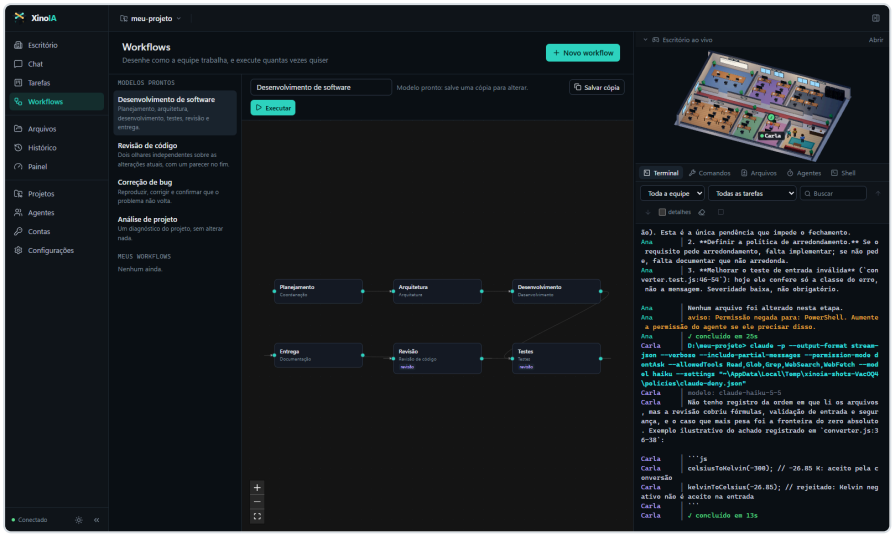
In parallel, your folder changes only when you choose **Integrar** (merge) at the end, or **Descartar** (discard). If two tasks touch the same spot, a new task resolves the conflict. A worktree starts without `node_modules`, so the agent must install dependencies to run tests in it. Merging requires that you have no uncommitted changes, so nothing of yours gets overwritten.

## Report

At the end, the coordinator writes a report with what was done, the decisions and what's pending.

> **Practice**
> In a test folder, ask: "Create a function that converts Celsius to Fahrenheit, with tests". Approve the plan and watch the handoff in the office.

# 8. Workflows




The visual workflow editor and the ready-made templates.

A workflow is a process you design once and use as often as you like. The templates cover software development, code review, bug fixing and project analysis. To change a template, use **Salvar cópia** (save a copy).

| In each step | Meaning |
|---|---|
| **Título** and **O que fazer** (title, what to do) | The request. Be specific, and say how to know it's finished. |
| **Função** (role) | The first agent on the team with this role runs it. |
| **Agente específico** | Fixes an agent, instead of choosing by role. |
| **Tipo** (type) | Work, or Review, which approves or rejects. |
| **Condição** (condition) | Always, only if the review approves, or only if it rejects. |
| **Tentativas em caso de falha** | How many attempts before stopping. |

Connect steps by dragging between them. A step starts only after the ones connected to it.

> **Tip**
> A read-only reviewer can't run commands. Have an earlier step gather the output of `git diff` and the tests, so the reviewer has something to evaluate.

## Goal queue

In **Fila** (queue), give a workflow a list of goals. Each goal becomes a mission, and the next one starts when it ends.

| Option | What it does |
| --- | --- |
| **Se um objetivo falhar** (if a goal fails) | **Parar e esperar por mim** (stop and wait) leaves the mission paused for you to fix. **Registrar e seguir** (log and move on) cancels the mission, records why and goes to the next one. |
| **Limite de gasto** (spending limit) | A token total, checked between goals. If any run doesn't report usage, the queue stops. |

With uncommitted changes, the queue doesn't start the goal and names the files: end the workflow with a commit step, or do it yourself and click **Continuar**. Outside Autônomo controlado, each goal waits for plan approval. Usage limits and sign-in problems are never skipped.

# 9. Execution modes

In **Configurações** (Settings). In automatic runs, tools can't stop midway to ask, so protection comes from limits set in advance.

| Mode | Plan | Tasks | Full permission | Destructive command |
|---|---|---|---|---|
| **Supervisionado** (supervised) | You approve | You release each | Limited to edit | Stops and asks you |
| **Semiautônomo** (semi, default) | You approve | Run by themselves | Limited to edit | Stops and asks you |
| **Autônomo controlado** (controlled autonomous) | Can be automatic | Run by themselves | Allowed | Only logs a warning |

Settings also holds two limits: minutes without output before an agent counts as stuck, and the maximum time for a run.

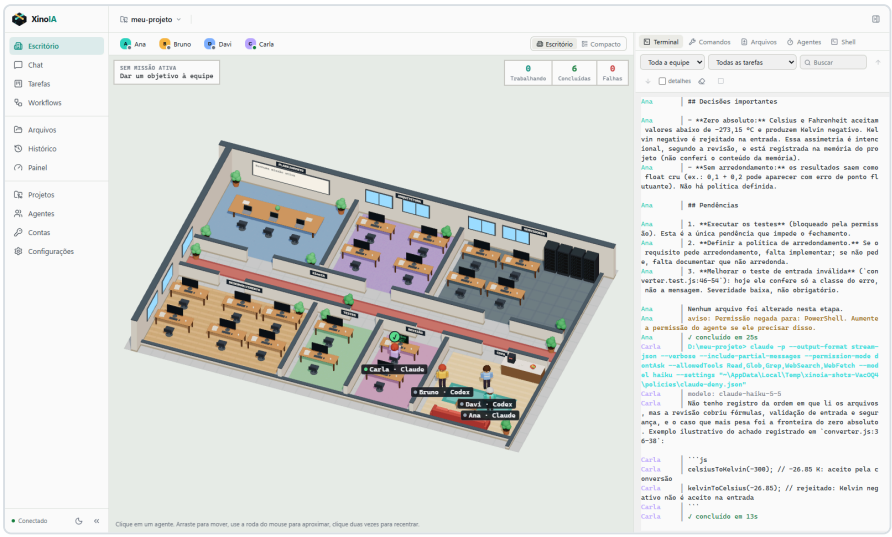
> **Detection is a safety net, not a guarantee**
> It looks for patterns such as deleting whole folders or discarding Git changes, but it doesn't see everything. Worktree isolation separates work in Git and isn't an operating system sandbox: an agent on "full" can reach the rest of the computer.

## Which to choose

- **While learning, or on an important project:** supervised.
- **Day to day:** semi-autonomous.
- **When you need "full":** controlled autonomous, briefly, with a commit first, then switch back.

# 10. Following the work

## Office




The office in the light theme, with available agents in the break room.

Everything shown comes from real events: an agent given work walks to its desk and starts typing, the monitor lights up, a bubble shows the real command, and whoever delivers a task carries the document to a colleague. Available agents stay in the break room.

- **Click an agent** to see its task, terminal and history, and to use **Conversar** (talk) or **Interromper** (stop).
- **Drag** to move, use the **mouse wheel** to zoom, and double-click to recenter.
- The **scoreboard** shows the mission, tasks and real counters. The whiteboard in the room shows the mission in progress.
- **Escritório ao vivo** (live office): the small view above the terminal, on other screens.

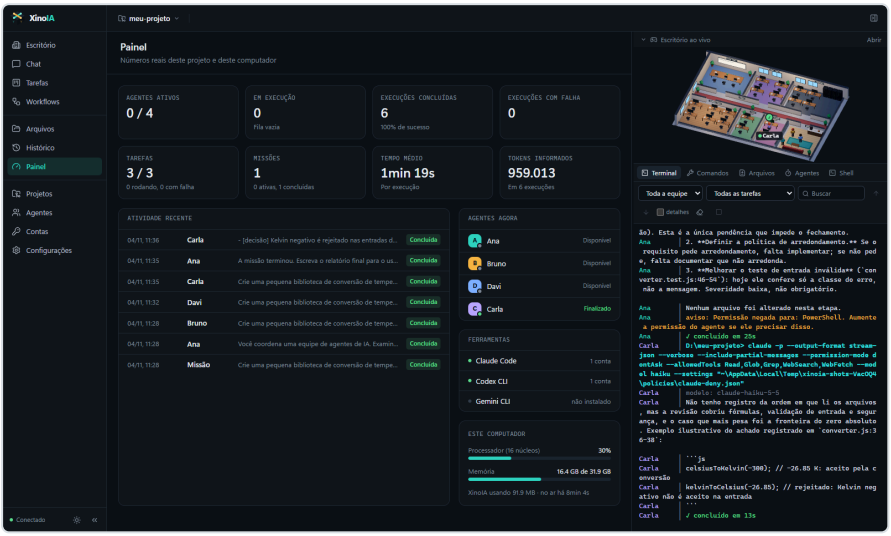
> **Saving your graphics card**
>
> The scene uses WebGL, at no more than 30 frames per second. Next to a heavy 3D program, choose **Compacto** (compact) and collapse the small view: the scene stops being drawn, and the same information appears in a table. Real graphics card usage hasn't been measured yet.

## Right panel

| Tab | What it shows |
|---|---|
| **Terminal** | The real output of each tool, for the team or one agent, searchable. |
| **Comandos** (commands) | The commands run, and which failed. |
| **Arquivos** (files) | The files changed, and by whom. |
| **Agentes** (agents) | Each one's state, and tokens reported by the tools. |
| **Shell** | A real terminal in the project folder, for your own commands. |

## Other screens

- **Arquivos** (files): view and edit project files, and what changed since the last commit.
- **Histórico** (history): every run in the project, filtered by agent and state.
- **Painel** (dashboard): real numbers about the project and the computer.




The dashboard, with runs, times and reported tokens.

# 11. Integrations, MCP and Unreal Engine

In **Contas > Integrações**, add skills, plugins and MCP servers. Then turn the integration on for each agent that will use it. XinoIA shows what it found on this computer. Configurations with secrets aren't copied, and stay available to the computer's login.

## Unreal Engine

1. In Unreal Editor, enable the **ModelContextProtocol** and **AllToolsets** plugins, and run `ModelContextProtocol.StartServer` in the console.
2. In Claude Code, install the official skills plugin.
3. In Contas > Integrações, use **Adicionar Unreal Engine** and check for "Respondendo" (responding). The default address is `http://127.0.0.1:8000/mcp`.
4. In Agentes, turn on the integration, at "edit" or "full".

| Rule | Why |
|---|---|
| **Tasks at once: 1** | MCP acts on the open editor, which stays in the original folder, not in the worktree. Maps and assets are binary, and an agent has no way to merge a conflict between them. |
| **One agent at a time in the editor** | The integration already works this way. More agents don't make this work faster. |
| **Compiling needs "full"** | The build writes to `%LOCALAPPDATA%`, outside the protected area of "edit". |
| **Check if the editor must be closed** | Some builds and tests don't run with it open. In that case, leave the integration off for those tasks. |
| **Save before switching sessions** | Save maps in the editor and check with `git status`. |

> **Known limit**
> Plugin loading and configuration are tested, but tool calls against an open editor haven't been tested by XinoIA yet. Start with a small change that's easy to undo.

# 12. Usage: how to spend less of your limit

XinoIA shows usage and limits only when a tool reports them, and never invents numbers. See them in **Contas** (limit windows, when the CLI reports them) and as reported tokens in the dashboard and terminal.

Usage varies a lot. In a real project, each Codex task used from 30 thousand to nearly 600 thousand input tokens: most of it is the agent reading the project for context.

- **Small tasks** with a clear definition of done spend less than a vague request.
- **Say where to look:** naming the files keeps the agent from searching the whole project.
- **Choose model and effort:** simple tasks don't need the highest level.
- **Limit fix rounds**, and use `Equipe, ...` only when you need to hear everyone.
- **Pause** if a mission goes the wrong way, and guide the team.

> **What to do at a limit**
> The task stops with a clear message. Wait for it to renew, change the agent's account or reassign the task. With the option on, it can pass to an agent from another provider.

# 13. Moving a project into XinoIA

Files stay where they are, and the login is the computer's. **Conversations don't move**: XinoIA opens new sessions, so everything important must be written in a file.

1. **Stop the current work.** Never let another session and XinoIA change the same folder at the same time.
2. **Save a point in Git.** Decide what to do with pending changes.
3. **Write** `XINOIA_HANDOFF.md` at the root: goal, current state, work in progress, next steps, decisions, how to build and test, environment and pitfalls. Never include passwords or keys.
4. **Check the rules:** Codex reads `AGENTS.md`, not `CLAUDE.md`.
5. **In XinoIA:** check the accounts, add the project, record the memory, add the integrations and create the team.
6. **Test without changing anything:**

```
@Coordination read XINOIA_HANDOFF.md and tell me, in five lines, the state of the project
and the next step. Don't change any file.
```

Then create a small mission, with **Tarefas ao mesmo tempo** at 1:

```
Read XINOIA_HANDOFF.md and continue with the first item under "Next steps". Do only that
item, prove it works as the file describes, and update XINOIA_HANDOFF.md with what changed.
```

Another AI can prepare this by following `docs/GUIA_PARA_IAS.md` in the repository. For security, another AI can't control the installed app: the clicks are yours. With `npm run verify`, it can check the system without accounts or quota.

# 14. Security and privacy

## What XinoIA does

- The local server listens only on `127.0.0.1`, and requires a new token at each start.
- Sign-in happens in each tool's official flow. It doesn't read the access files.
- An API key stays encrypted by Windows, and never returns to the screen.
- Prompts go through standard input, never as command-line arguments.
- Known secrets and your home folder are masked in events and logs.
- Updates are installed only after an Ed25519 signature check.

## What it doesn't do

- At the "full" level, an agent runs commands with the same access as your account.
- Content an agent reads may try to trick it (prompt injection). Be careful with projects of unknown origin.
- Local data is protected by your system account, without encryption of its own.

## Where your data goes

| What | Where | When |
|---|---|---|
| Prompts and files the agent reads | The vendor of the chosen tool | When an agent works |
| Update check | GitHub | On open and every 4 hours. Can be turned off |
| Installer and sign-in | The vendors' official sites | Only when you click |

> **Three habits that prevent almost every problem**
> Use Git and commit before a mission. Start with low permissions. Review before merging.

# 15. Troubleshooting

| Message or symptom | What to do |
|---|---|
| "A conta precisa de login" (account needs sign-in) | In Contas, **Entrar pelo navegador**. The official login can expire. |
| "O provedor recusou a chave de API" (key refused) | The key is revoked, wrong or out of credit. Use **Trocar chave** (change key). |
| "Limite de uso da conta" (usage limit) | Wait for it to renew, change the agent's account or reassign the task. |
| "Permissão negada para: Bash" (permission denied) | The agent tried a command it isn't allowed to run. See chapter 5. |
| Command rejected with "blocked by policy" in Codex | This happened in separate profiles on version 0.1.0. Update. In a new profile, the first command can be slow. |
| Build fails with access denied in `AppData` | The protected area doesn't write outside the project. See "When edit isn't enough" in chapter 5. |
| "A ferramenta não está instalada" (tool not installed), but it is | Close and reopen XinoIA, or restart Windows, so `PATH` updates. |
| Merging says there are uncommitted changes | Commit or stash your changes. This protects your work. |
| The agent says it can't run tests | Use an agent allowed to do so. In worktrees, it also must install dependencies. |
| Retomar makes the mission stop again | There's a rejected review. Open the task: **Tentar de novo** or **Pular**. |
| Unreal MCP shows "Sem resposta" (no response) | Open the editor, start the server and check the address. |
| The office doesn't open in 3D | The computer doesn't offer WebGL. Use the **Compacto** view. |
| Database is from a newer version | You opened data from a newer version in an older one. Install the latest. |
| Damaged database | XinoIA keeps the file aside, restores the latest backup and warns you in Configurações > Dados. |

# 16. Hands-on path: from zero to a team

1. Connect one AI through the browser and check it in Contas.
2. Create a project with **Pasta nova** and make the first commit in **Shell**: `git init`, `git add -A`, `git commit -m start --allow-empty`.
3. Create the team from chapter 5.
4. Talk to an agent, and watch the run in the terminal.
5. Create a small mission, edit the plan and approve it.
6. Let a review reject, and watch the automatic fix.
7. Save a copy of a workflow, change a step and run it.
8. Run a mission with **Tarefas ao mesmo tempo** at 2, and merge the result.
9. Open the dashboard, and compare the tokens of each run.